part1

1

GCC 全称 GNU Compiler Collection，即 GNU 编译器集合。它最初是 GNU C Compiler，后来发展为支持多种语言的编译器套件，包括 C、C++、 Fortran、Go 等。

GCC 的作用：把源代码（如 .c 文件）编译成计算机能执行的可执行文件（如 .exe 或 Linux 下的可执行文件）。它负责预处理、编译、汇编、链接四个 阶段。

2

MinGW 全称 Minimalist GNU for Windows，即 Windows 上的极简 GNU 工具链。因为 GCC 原生主要运行在 Linux/Unix 环境，Windows 不能直接使 用，MinGW 把 GCC、链接器、汇编器以及 Windows 头文件和库移植到 Windows，让你可以在 Windows 上使用 GCC 编译出原生的 Windows 程序。

现在更常用的是 MinGW-w64，支持 32 位和 64 位 Windows。

MinGW 的作用：在 Windows 上提供一套类似 Linux 的 GCC 编译环境，把 C/C++ 代码编译成 Windows 可执行文件。它内部包含 gcc.exe、g++. exe、gdb.exe 等工具。

3

GCC 是编译器本身。

MinGW 是 Windows 上的 GCC 发行版/工具链，里面包含 GCC。

在 Windows 上安装 MinGW 后，就可以使用 gcc 命令；在 Linux 上通常直接安装 GCC，不需要 MinGW。

{ // 使用 IntelliSense 了解相关属性。 // 悬停以查看现有属性的描述。

```
"version": "0.2.0",
"configurations": [
    {
        "name": "gcc.exe - 生成和调试活动文件",  // 该调试任务的名字，启动调试时会在待选列表中显示

        "type": "cppdbg",                          // 调试器类型，C/C++ 使用 cppdbg

        "request": "launch",                       // 请求类型：launch 表示启动新程序

        "program": "${fileDirname}\\${fileBasenameNoExtension}.exe", // 要调试的可执行文件路径

        "args": [],                                // 传给程序的命令行参数

        "stopAtEntry": false,                      // 是否在程序入口处暂停（false 表示
```

```
直接运行）

        "cwd": "${workspaceFolder}",             // 程序运行的工作目录

        "environment": [],                       // 环境变量

        "externalConsole": false,                // 是否使用外部控制台：false=内置终
端，true=外部终端

        "MIMode": "gdb",                         // 调试器模式，使用 gdb

        "miDebuggerPath": "C:\\mingw64\\bin\\gdb.exe", // gdb.exe 的完整路径

        "setupCommands": [
            {
                "description": "为 gdb 启用整齐打印", // 命令描述

                "text": "-enable-pretty-printing",   // 发送给 gdb 的命令

                "ignoreFailures": true               // 失败是否忽略
            }
        ],

        "preLaunchTask": "C/C++: gcc.exe build active file" // 调试前先执行的编译任
务，对应 tasks.json 中的 label
    }
]
```

}

part2

1

变量的类型决定了：

变量在内存中占用的字节数

能存储的数据范围

能进行的操作（如整数可取模，浮点数不能）

数据的解释方式（同样的二进制位，按 int 和 float 解释结果完全不同）

C 语言是强类型语言，每个变量必须先声明类型。类型错误会导致编译错误或运行时未定义行为

存放年龄 18 应选择int

"apple" 是一个字符串，正确存储方式是使用字符数组或字符指针

2

数组下标从 0 开始

访问 arr[5] 或 arr[-1] 属于数组越界，是未定义行为（Undefined Behavior）。可能导致： 读取或修改其他变量的值、程序崩溃（段错误）、数据损坏、安全漏洞（如栈溢出攻击）

C 语言不检查数组边界，编译器不会报错，运行时也可能不报错， 越界访问可能悄悄修改其他内存，导致难以调试的 bug， 攻击者可以利用越界写入覆盖返回地址，劫持程序控制流。

3

for 循环 for (初始化; 条件判断; 迭代) { // 循环体 } 初始化：循环开始前执行一次，通常用于设置循环变量。

条件判断：每次循环前检查，为真则执行循环体，为假则退出。

迭代：每次循环体执行后执行，通常用于更新循环变量。

while 循环 while (条件) { // 循环体 } 先判断条件，为真则执行循环体，然后再次判断。

do...while 循环 do { // 循环体 } while (条件); 先执行一次循环体，再判断条件。至少执行一次。

while 与 do...while 的关键区别 while：先判断，可能一次都不执行。 do...while：先执行，至少执行一次。

int sum = 0;

for (int i = 1; i <= 10; i++) {

```
    sum += i;
```

}

printf("sum = %d\n", sum); // 55

int sum = 0;

int i = 1;

while (i <= 10) {

```
    sum += i;

    i++;
```

}

printf("sum = %d\n", sum); // 55

4

1. 逻辑表达式与算术表达式的本质区别 对比项 算术表达式 逻辑表达式 运算对象 数值（整数、浮点数） 布尔值（真/假）或可判断真假的表达式 运算结果 数值 布尔值（C 语言中为 0 或 1） 运算符 + - * / % && || ! 以及比较运算符 > < == != >= <= 用途 计算数值 控制程序流程（条件、循环）
2. 逻辑运算符含义 &&：逻辑与，两边都为真时结果为真。

||：逻辑或，至少一边为真时结果为真。

!：逻辑非，真变假，假变真。

#include <stdio.h> #include <string.h>

int main() { char name[100]; // 存储姓名，支持带空格的姓名 int age; // 存储年龄 char choice; // 存储用户是否继续的选择 int count = 0; // 记录读取-打印操作的次数

```
while (1) {
    // 读取姓名（使用 fgets 以支持包含空格的姓名）
    printf("请输入姓名: ");
    if (fgets(name, sizeof(name), stdin) == NULL) {
        break;  // 遇到 EOF 或错误，退出
    }
    // 去掉 fgets 读入的末尾换行符
    name[strcspn(name, "\n")] = '\0';

    // 读取年龄
    printf("请输入年龄: ");
    if (scanf("%d", &age) != 1) {
        // 输入不是整数，清空缓冲区并提示
        while (getchar() != '\n');
        printf("输入无效，请重新输入。\n");
        continue;
    }
    // 清空缓冲区中剩余的换行符
    while (getchar() != '\n');

    // 以原格式打印姓名与年龄
    printf("姓名: %s, 年龄: %d\n", name, age);

    // 操作次数加一
    count++;

    // 询问是否继续
    printf("是否继续输入？(y/n): ");
    scanf(" %c", &choice);   // 前面的空格跳过空白字符
    while (getchar() != '\n'); // 清空缓冲区

    if (choice != 'y' && choice != 'Y') {
        break;  // 用户选择不继续，退出循环
    }
}
```

```
    // 打印总操作次数
    printf("总共进行了 %d 次输入和打印操作。\n", count);

    return 0;
```

}

part3

#include <stdio.h>

// 乘方函数：计算 base 的 exp 次方

int power(int base, int exp) { int result = 1; for (int i = 0; i < exp; i++) { result *= base; } return result; }

// 计算综合成绩：传入三项成绩，返回综合成绩

int compute_composite(int a, int b, int c) { int avg = (a + b + c) / 3; // 平均分（整数除法）

```
    // 方差 = 各项与平均分之差的平方和 / 3

    int var = (power(avg - a, 2) + power(avg - b, 2) + power(avg - c, 2)) / 3;

    // 综合成绩 = 3 * 平均分 - 方差 / 3

    return 3 * avg - var / 3;
```

}

// 排序并打印：按综合成绩降序输出三人姓名 void sort_and_print(const char *name1, int score1, const char *name2, int score2, const char *name3, int score3) { const char *names[3] = {name1, name2, name3}; int scores[3] = {score1, score2, score3};

```
    // 冒泡排序（降序）
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (scores[j] < scores[j + 1]) {
                // 交换分数
                int tmp_score = scores[j];
                scores[j] = scores[j + 1];
                scores[j + 1] = tmp_score;
                // 交换姓名
                const char *tmp_name = names[j];
                names[j] = names[j + 1];
                names[j + 1] = tmp_name;
            }
        }
    }
```

```
    // 输出排序结果
    printf("%s > %s > %s", names[0], names[1], names[2]);
```

}

int main() { int x1, x2, x3; int y1, y2, y3; int z1, z2, z3;

```
    printf("请输入小明的三项成绩（顺序为A B C,以一个空格为间隔）：");
    scanf("%d %d %d", &x1, &x2, &x3);
    printf("请输入小强的三项成绩（顺序为A B C,以一个空格为间隔）：");
    scanf("%d %d %d", &y1, &y2, &y3);
    printf("请输入小林的三项成绩（顺序为A B C,以一个空格为间隔）：");
    scanf("%d %d %d", &z1, &z2, &z3);

    // 分别计算三人的综合成绩
    int zh1 = compute_composite(x1, x2, x3);
    int zh2 = compute_composite(y1, y2, y3);
    int zh3 = compute_composite(z1, z2, z3);

    // 排序并输出
    sort_and_print("小明", zh1, "小强", zh2, "小林", zh3);

    return 0;
```

}

a = 10，b = 20

C 语言默认采用值传递（Pass by Value）。当调用 swap(a, b) 时，发生的是：

系统在栈上为 swap 的形参 a 和 b 分配新的内存空间。

把 main 中实参 a 和 b 的值复制一份，分别赋给形参。

swap 函数内部操作的，是形参这份副本，与 main 中的原始变量完全无关。

函数返回时，形参的生命周期结束，副本被销毁。main 中的 a 和 b 从未被触及。